

Simulador P2P sobre sockets TCP

Alejandro Q

Angie E

Jose H

IC-6600 Principios de Sistemas Operativos
I Semestre 2026

1. Introduccion

Este documento describe el diseno e implementacion de un simulador P2P para busqueda y transferencia de archivos mediante sockets TCP en C. El sistema combina un servidor central de metadatos con un mecanismo distribuido de busqueda entre vecinos, de modo que los clientes pueden localizar archivos aun cuando la busqueda central no retorna resultados.

El proyecto se organiza alrededor de dos comandos de usuario. El comando **find** localiza archivos por coincidencia parcial de nombre y puede usar el servidor, la red distribuida o un modo automatico que intenta ambos. El comando **request S H** descarga un archivo identificado por tamano y hash desde los pares encontrados, dividiendo el rango de bytes entre los duenos disponibles y ensamblando el resultado en la carpeta compartida local.

La solucion busca cumplir tres objetivos practicos: mantener un protocolo de comunicacion estable, tolerar fallas comunes como desconexiones y retiro de la carpeta compartida, y validar el comportamiento con pruebas unitarias e integraciones locales de varios clientes.

2. Marco teorico

2.1. Redes P2P

En una red peer-to-peer cada nodo puede actuar como cliente y servidor. Esta propiedad permite que un cliente solicite datos a otros clientes sin depender de un unico repositorio de archivos. El proyecto usa un servidor central solo para registrar metadatos y sembrar vecinos iniciales; las transferencias de datos y la busqueda distribuida ocurren directamente entre clientes.

2.2. Calidad de servicio

La calidad de servicio se refleja en tiempos de busqueda, disponibilidad de pares y continuidad de las transferencias. La implementacion reduce bloqueos separando responsabilidades en hilos: la consola queda en el hilo principal, el puerto de datos tiene un listener dedicado y cada transferencia usa hilos de segmento. Tambien se ignora **SIGPIPE** y se revisan errores de I/O para evitar que una conexion cerrada termine el proceso.

2.3. Hashing de archivos

El sistema identifica un archivo por el par (**hash**, **size**). Se usa una funcion propia basada en FNV-1a de 64 bits sobre el contenido del archivo, lo que permite reconocer el mismo archivo aunque cambie su nombre. El tamano se incluye como segunda llave para disminuir falsos positivos y para dividir los rangos de descarga.

2.4. Sockets TCP y framing

Todos los mensajes viajan sobre TCP. Antes de cada mensaje se envia una longitud externa con `net_send_msg`; dentro de ese frame aparece un `p2p_msg_header_t` con opcode, version y tamaño del payload. Los enteros de multiples bytes se codifican en orden de red con `htons`, `htonl` y conversiones de 64 bits equivalentes.

2.5. Busqueda distribuida

La busqueda distribuida usa flooding con TTL. Cada consulta incluye un ID de 16 caracteres hexadecimales, termino de busqueda, IP y puerto del originador, y un contador TTL. Cada cliente mantiene una cache de IDs vistos durante 60 segundos para evitar ciclos. Cuando recibe una consulta nueva, revisa su carpeta compartida, responde directamente al originador si encuentra coincidencias y reenvia a sus vecinos con TTL decrementado.

El TTL por defecto es 4 y el maximo aceptado es 6. Con grado promedio 3, un TTL 4 tiene un limite teorico de aproximadamente $3^4 = 81$ mensajes, suficiente para una prueba local de tres clientes sin saturar la LAN. La ventana de agregacion por defecto es 2 segundos y puede configurarse al iniciar el cliente.

3. Descripcion de la solucion

3.1. Arquitectura

La solucion esta dividida en modulos. `common/` contiene el protocolo, wrappers de sockets y hashing. `server/` implementa el servidor central y su registro de pares. `client/` contiene el arranque, escaneo de carpetas, API hacia el servidor y REPL. `transfer/` implementa el listener del puerto de datos, el envio de rangos y el receptor que ensambla segmentos. `search/` contiene vecinos, flooding y agregacion de respuestas.

3.2. Servidor central

El servidor escucha conexiones TCP y crea un hilo por cliente aceptado. Cada mensaje debe tener version valida, opcode conocido y longitud de payload correcta. El opcode `REGISTER` registra la IP observada en la conexion, el puerto de datos anunciado y la lista de archivos del cliente. La respuesta incluye hasta `P2P_MAX_NEIGHBORS` pares recientes, excluyendo al par que acaba de registrarse. El opcode `FIND` busca coincidencias parciales por nombre y retorna metadatos S, H, IP, puerto y nombre. Los errores de version, formato u opcode se responden con `ERROR`.

3.3. Cliente y comandos

Al iniciar, el cliente recibe IP y puerto del servidor, puerto de datos y carpeta compartida. Escanea recursivamente la carpeta, calcula hash y tamaño, registra los metadatos en el servidor, inicializa la busqueda distribuida y arranca un listener unico para mensajes entre pares.

El REPL soporta:

- `find -s nombre`: busqueda centralizada.
- `find -d nombre`: busqueda distribuida.
- `find nombre`: servidor primero; si falla o retorna cero resultados, usa busqueda distribuida.
- `request S H`: descarga usando los pares de la ultima busqueda mostrada.

3.4. Transferencia segmentada

El receptor filtra los resultados cacheados por **S** y **H**, elimina pares duplicados y divide el archivo entre los pares disponibles. Cada hilo de segmento abre una conexión TCP al puerto de datos del par, envía **TRANSFER_REQ** con **hash**, **size**, **byte_start** y **byte_end**, y recibe frames **TRANSFER_DATA**. Los bytes se escriben con **pwrite** en un archivo parcial. Si todos los segmentos terminan bien, el archivo parcial se renombra a **download_S_H.bin**.

3.5. Listener unico del puerto de datos

Para evitar que transferencia y búsqueda distribuida compitan por el mismo puerto, el cliente usa un solo listener. El dispatcher lee el header del frame y enruta por opcode: **TRANSFER_REQ** hacia el sender, **QUERY_FLOOD** hacia la lógica de flooding, **QUERY_RESULT** hacia el agregador, y opcodes inválidos hacia **ERROR**.

3.6. Búsqueda distribuida

La lista de vecinos se siembra con la respuesta de registro del servidor y se actualiza con resultados observados. **find -d** genera un ID de consulta de 64 bits en hexadecimal, marca el ID como visto localmente, envía **QUERY_FLOOD** a los vecinos y abre una ventana de agregación. Cada par que recibe una consulta nueva escanea su carpeta compartida en ese momento; por eso también puede encontrar archivos agregados después del registro central. Las respuestas **QUERY_RESULT** se envían directamente al originador.

Se mantuvo congelado **common/protocol.h**. Como el mensaje congelado incluye el originador pero no el puerto de datos del remitente inmediato, el control de ciclos depende de la cache de IDs vistos. En las pruebas locales, esto permite trabajar con varios clientes en la misma máquina sin confundir pares distintos que comparten IP.

3.7. Tolerancia a hot-unplug

Antes de escanear o abrir archivos se revisa acceso a la carpeta. Si la carpeta compartida desaparece, el cliente registra una advertencia y continúa vivo. Las fallas de lectura se propagan como errores de transferencia o como búsquedas locales sin resultados, pero no terminan el servidor ni el proceso cliente.

4. Descripción de la experiencia

El desarrollo se trabajó por capas. Primero se estabilizaron las estructuras de protocolo, la función de hash, los wrappers de red y las pruebas unitarias. Con esa base, el siguiente bloqueo fue el servidor central: hasta implementar **REGISTER** y **FIND**, el cliente podía escanear y enviar mensajes pero no existía una respuesta real de integración.

El reto principal fue mantener un contrato de red consistente. Cada módulo debía respetar el framing externo, el header interno y el orden de red para enteros de 16, 32 y 64 bits. Se agregaron pruebas enfocadas para el handler del servidor, incluyendo desconexión temprana y frames malformados, porque esos casos son comunes cuando hay varios procesos en pruebas.

La segunda decisión importante fue unificar el listener del puerto de datos. Inicialmente transferencia y búsqueda distribuida podían interpretarse como listeners separados, pero ambos debían usar el mismo puerto del cliente. La solución fue un dispatcher por opcode, que conserva la transferencia existente y agrega los mensajes de búsqueda sin crear una carrera por el bind.

La validacion final se hizo con un smoke local de tres clientes. Dos pares compartieron el mismo archivo para comprobar descarga segmentada multi-peer; un tercer par ejecuto `find -s`, `find -d`, `find` con fallback, y `request S H`. Tambien se agrego un archivo despues del registro central para demostrar que la busqueda distribuida reescanea la carpeta viva. Por ultimo, se elimino una carpeta compartida mientras los procesos seguian activos para verificar tolerancia a hot-unplug.

5. Aprendizajes personales

Integrante 1

El trabajo del servidor mostro la importancia de congelar temprano el protocolo. Un cambio pequeno en un struct o en el orden de bytes afecta a cliente, transferencia, busqueda y pruebas. Tambien reforzo la necesidad de validar entradas malformadas sin asumir que todos los clientes se desconectan de forma limpia.

Integrante 2

La parte de cliente y transferencia dejo claro que la concurrencia debe tener limites simples. Separar REPL, listener y segmentos de descarga hizo que el sistema siguiera respondiendo mientras ocurren I/O de red y disco. El uso de `pwrite` y archivos parciales facilito ensamblar segmentos sin depender del orden en que terminan los hilos.

Integrante 3

La busqueda distribuida evidencio el balance entre alcance y control de propagacion. TTL, cache de consultas vistas y ventana de agregacion son decisiones pequenas pero determinan si la red encuentra resultados o se satura. Tambien fue valioso documentar las pruebas mientras se implementaban, porque el comportamiento distribuido es dificil de explicar solo leyendo codigo.

6. Conclusiones

El simulador cumple el flujo principal solicitado: los clientes registran sus archivos en un servidor central, buscan por nombre, descargan archivos por hash y tamano desde uno o varios pares, y pueden recurrir a una busqueda distribuida cuando el servidor no entrega resultados. Las pruebas locales con tres clientes demostraron busqueda centralizada, busqueda distribuida, fallback automatico, descarga segmentada y tolerancia a la desaparicion de una carpeta compartida.

La decision de usar TCP para todos los mensajes simplifico la confiabilidad del transporte y permitio reutilizar el mismo framing. El costo es que cada propagacion abre conexiones, por lo que el TTL debe mantenerse bajo. Para la red de prueba, TTL 4 fue suficiente; subirlo a 6 solo tendria sentido en una red mas grande y con mediciones de carga.

La principal limitacion es que el servidor central no elimina automaticamente pares que ya no estan disponibles. Si un cliente se apaga sin avisar, sus metadatos pueden permanecer hasta que falle una transferencia o una busqueda distribuida encuentre alternativas. Como mejora futura se agregarian heartbeats o expiracion de pares en el registro central, junto con una opcion explicita de refrescar metadatos despues de cambios en la carpeta compartida.

Otra mejora seria enriquecer el protocolo distribuido con el puerto de datos del remitente inmediato. La version actual conserva el protocolo congelado y usa la cache de IDs vistos para evitar

ciclos, lo cual funciona para la prueba, pero un campo de remitente permitiría excluir con más precisión el enlace por el que llegó una consulta.